Assessing Security Risks in Low-Code Development Platforms: A Systematic Literature Review

2024 International Conference on Advances in Technology and Computing (ICATC) | 979-8-3315-1821-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICATC64549.2024.11025368

Hiruni Hathnagoda
dept. Industrial Management
Faculty of Science
University of Kelaniya
Kelaniya, Sri Lanka
hirunih99@gmail.com

Ruwan Wickramarachchi
dept. Industrial Management
Faculty of Science
University of Kelaniya
Kelaniya, Sri Lanka
ruwan@kln.ac.lk

***Abstract*— Low Code Development Platforms (LCDPs) have transformed software development by enabling rapid application creation with minimal coding, democratizing the process beyond professional developers. This type of democratization comes with considerable security risks. The paper aims to classify security risks associated with seven leading LCDPs: Mendix, OutSystems, Microsoft Power Apps, ServiceNow, Salesforce, Oracle, and Pegasystems. It evaluates respective mitigation strategies to improve security. 27 studies reviewed using the PRISMA methodology showed critical vulnerabilities in application, information, platform specific, and user related risks. The most frequent application-level vulnerabilities are code injection, XSS, and insecure APIs, which amateur developers unaware of secure coding practices further worsen. Common information security risks across various platforms include weak encryption, data breaches, and misconfigurations. User related risks include weak credentials and insider threats that expose applications to breaches in Salesforce and Oracle. With automated patch management and secure coding practices, Mendix and OutSystems have better security postures. In contrast, others like ServiceNow and Pegasystems remain vulnerable because of delayed patching and other manual security processes, which introduces a higher probability of human error. Based on this, the study concluded that an extended multi dimensional security strategy is required, comprising technological solutions such as automation, human centered interventions, training, and user management in managing the risks within LCDPs.**

Keywords—Citizen Developers, Low-Code Development Platforms, Low-Code Software Development, Mitigative Measures, Security Risks

I. INTRODUCTION

Low Code Development Platforms have irreversibly altered the software development landscape, introducing a paradigm shift that radically enhances productivity and usability for target end-users, including non-developers. They reduce reliance on heavy manual coding by providing visual development interfaces together with preconfigured modules that enable a wider range of users than professional developers to develop and deploy applications much faster [1]. As organizations increase their focus on agility and rapid time-to-market for digital solutions, the adoption of LCDPs has surged, with global spending on such platforms forecasted to reach more than $26 billion in 2024, up 19.6% from year-earlier levels. It reflects the fast growth rate and exactly how main LCDPs have become in current software development, especially regarding enterprise software, CRM, BPM, and mobile application development, where they facilitate processes and democratize software development [2].

At the same time, multiple challenges are associated with the wide diffusion of LCDPs. The same reasons that make them so appealing, such as ease of use and availability to non-professional developers, also become significant contributors to security vulnerabilities [3]. As more users with limited technical expertise adopt these platforms, the likelihood of security vulnerabilities increases. This calls for comprehensive risk assessments. Moreover, the literature underlines that only powerful mitigative measures can ensure integrity, confidentiality, and availability for applications developed on LCDPs. Despite such a trend of growing awareness, the current research still lacks considerations for the long-term security implications of using LCDPs, especially in a non-professional developer setting. In light of this, the current study aims to close this gap by thoroughly analyzing the security threats and assessing how well the countermeasures that have been put in place have worked.

By the end of 2024, low-code development will make up almost 65% of application development labor, says Gartner, where the need for urgent security frameworks to be built for such platforms is felt [1]. Ironically, making LCDPs available to non-experts sometimes leads to security lapses and vulnerabilities. For this reason, assessing the specific security risks that this type of platform presents is paramount. Therefore, this scoping review is most appropriate for this research, since a wide exploration of the literature will be very useful in establishing suitable security frameworks that are tailored to the specific LCDP vulnerabilities that are not suitably covered by the generic security practices [4].

The key research objectives of the given work would therefore include the following: systematic identification of security vulnerabilities associated with LCDPs in a non-professional developer setup, analysis of the effectiveness of the existing security frameworks and tools to mitigate these risks, and development of a comprehensive set of best practices that can be adopted by an organization to improve the state of security. It targets the gap between the rapid application deployment capabilities of LCDPs and efficient security measures to ensure the secure deployment of low-code applications within varied industries.

The prominent LCDPs, identified by the Gartner Magic Quadrant 2023, will be covered in this research. These include Mendix, OutSystems, Microsoft Power Apps, ServiceNow, Salesforce, Oracle, and Pegasystems. These have been chosen in terms of market demand and ranking; thus, this will highlight the importance these carry. Based on their unique positioning in the Gartner Magic Quadrant 2023, where Oracle is in the category of Challengers and Pegasystems falls

979-8-3315-1821-9/24/$31.00 ©2024 IEEE

into the category of Visionaries, have chosen Oracle and Pegasystems for the research along with the Leaders.

With its ability to perform large-scale, complex deployments that boast robust cloud service and enterprise application integrations, Oracle is a strong Challenger in the LCDP space with a good potential to lead. Its low-code environments are excellent in data encryption, identity management, and threat detection, driven by AI-powered analytics and truly innovative security frameworks [5]. These surely place Oracle in a perfect position for rapid growth in the low-code market. With its AI-driven predictive security and adaptive workflows, the company's growth trajectory shoots up.

Pegasystems is positioned as a Visionary, applying AI and machine learning to drive automation in business processes. The company provides an extremely versatile and scalable low-code platform that caters to state-of-the-art security challenges for these applications. Indeed, research has shown that Pegasystems uses case management capability in such a manner that will place it well for market leadership in the future [6]. The trend of development accelerates with its AI-driven predictive security and adaptive workflows; it promises to be secure and a forward-looking platform.

II. Methodology

This systematic review uses the PRISMA methodology to rigorously assess security threats and mitigation strategies in Low Code Development Platforms. Information used here is available on major academic databases: Google Scholar, ScienceDirect, IEEE Digital Library, Springer Link, Wiley, and Emerald. "Low Code Development Platforms", "LCDPs", "Security risks", "Mitigative measures", and "Software Development" are the keywords used for the reference search. Appropriate search operators such as "AND" or "OR" were used to link the keywords to the needed filtering of the search. Further, additional platform-specific searches were also made through Mendix, OutSystems, Microsoft Power Apps, ServiceNow, Salesforce, Oracle, and Pegasystems to narrow down the findings. This will also allow capturing general and platform-specific security risks, which is important considering the features and architectures these LCDPs come in. A total of 21,327 documents were initially retrieved; after title and abstract screening, 27 relevant documents were selected for further review.

TABLE I. Inclusion And Exclusion Criteria

Inclusion Criteria (IC)	IC1	Publications from 2018 to 2024.
	IC2	Publications written in English.
	IC3	Peer-reviewed journal articles, conference papers, and book chapters.
	IC4	Studies focusing on LCDP security risks and mitigative measures.
	IC5	Platform-specific studies covering Mendix, OutSystems, Microsoft Power Apps, ServiceNow, Salesforce, Oracle, and Pegasystems.
Exclusion Criteria (EC)	EC1	Research not written in English.
	EC2	Research published before 2018.
	EC3	Non-peer-reviewed articles or opinion pieces.
	EC4	Studies not relevant to LCDP or that focus on general software security.
	EC5	Studies related to unrelated sectors or domains not concerning LCDPs.

Although the review has considered English language publications from 2010 to 2024, it has placed a greater emphasis on the more current works ranging from 2018 to 2024 to understand the LCDPs in their present status. The considered timeframe is since Low-Code Development Platforms have received much more attention in recent years, with greater implementation starting around 2018. The target publications included peer-reviewed journal articles, conference papers, book chapters indexed in SCOPUS, and reliable white papers. Non-peer-reviewed articles, opinion pieces, and editorials were also excluded since these failed to provide proper methodological approaches. Any related studies that focused on fields or sectors unrelated to LCDPs, where insights could not be transferred, were also excluded to keep the focal point on the security issues surrounding the platforms.

First, title and abstract screening were done, followed by a full-text review to narrow the articles further. The selection of sources will be based on the standardized form that guides reviewers to review the sources, state the discrepancies observed, and discuss these among reviewers. In case of disagreement, when the three reviewers have differing opinions, consultation would be sought from a third party to ensure that the selection is not biased. This led to a total of 6 studies that directly addressed overall security risks in LCDPs. Of the remaining 21, security risks and mitigative measures specific to the 7 platforms were discussed, emphasizing their usage by nonprofessional developers.

The review, through its comprehensively justified methodology, thus ensured the internally consistent and transparent manners of identifying security vulnerabilities and developing mitigative strategies for LCDPs, therefore offering a comprehensive landscape understanding and disclosing the prime importance of rigorous security frameworks that should be fitted within these platforms.

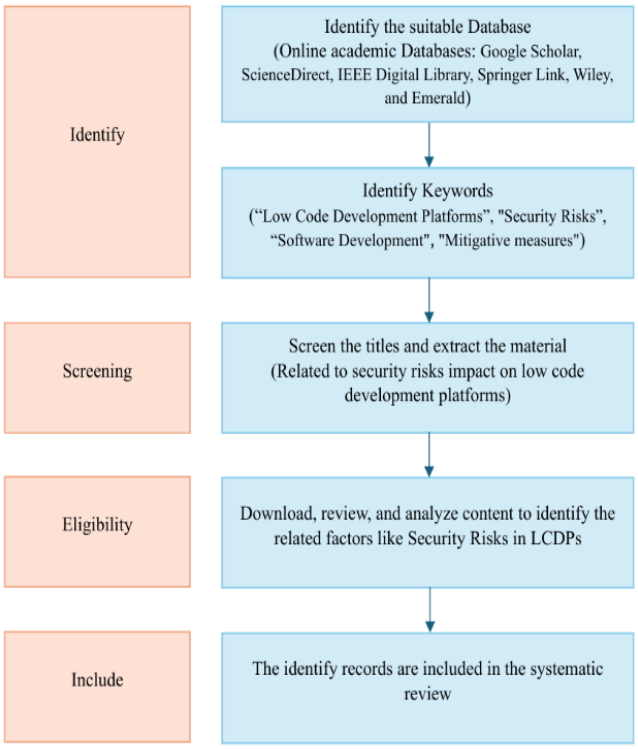

Fig. 1. PRISMA Framework.

III. Results and analysis

TABLE II. Low-Code Development Platform Security Risks

Category	Security Risk	Reference
Application Security Risks	Code Injection	[7], [8]
	Cross-Site Scripting (XSS)	[9], [10]
	Cross-Site Request Forgery (CSRF)	[9], [11]
	Insecure API Usage	[7], [12], [13], [14], [9]
	Improper Authentication	[7], [15], [16], [13], [12], [8], [17], [18], [19], [9]
Information Security Risks	Data Breaches	[7], [15], [20], [12], [21], [22], [11], [23], [8], [14], [9]
	Insufficient Data Protection	[7]
	Weak Encryption and Data Loss Prevention (DLP) Issues	[24], [22], [25]
	Insecure API Integrations	[12]
Platform-Specific Risks	Misconfigurations	[12], [25], [13], [21], [14], [8], [11]
	(Third-Party Integration Vulnerabilities)Vulnerabilities in External Libraries	[13], [18], [23]
	Vendor Lock-In	[14], [26]
	API Vulnerabilities	[7], [12], [13], [14], [9]
User-Related Risks	Weak User Credentials	[19]
	Lack of Security Awareness	[7]
	Privileged User Abuse	[20]
	User Mismanagement (Multiple Login Session)	[23], [27]
	Human Errors	[14], [18]

In recent years, LCDPs have transformed software development by making it possible for a large audience to create and deploy apps quickly. This developing accessibility has also created several security risks that will be discussed within the following four main categories: Application Security Risks, Information Security Risks, Platform-Specific Risks, and User-Related Risks. It makes concrete comparisons of those risks across a variety of platforms and points to different strengths and weaknesses regarding their security approaches. Classifying these identified security risks into the above-mentioned four distinct classes gives a more organized way of understanding and mitigating the risks, each class concerning another type of challenge emanating from another aspect of LCDPs.

A. Application Security Risks

Application security risks are the most critical in LCDPs, especially because of the involvement of non-professional developers who might not possess the technical skills required to make their applications secure. The most common risks identified from this category were code injections, cross-site scripting, cross-site request forgery, insecure API usage, and improper authentication. These are made worse by the ease of creation that LCDPs provide, as users might avoid dealing with the security implications by using a method other than standard coding.

These include code injection vulnerabilities, which are rated as a high threat since attackers can inject malicious codes into applications by exploiting insecure input validation and other lousy coding practices. Non-professional developers contribute to these risks since most of them do not adhere to strict secure coding guidelines. This is especially true for code injection attacks, commonly seen in platforms such as Microsoft Power Apps and Salesforce [7],[8]. Cross-site scripting and cross-site request forgery vulnerabilities tend to be overt in web-based applications. Such weaknesses offer an easy avenue through which attackers can inject malicious scripts into web pages to deceive users into performing something that they do not intend to do, thereby breaching the application's security.

In particular, Cross-Site Scripting is common in systems like Microsoft Power Apps or Salesforce since the focus on rapid application development leaves output encoding weak or nonexistent. This leaves applications open to an attacker who can inject malicious scripts into web pages viewed by other users, putting sensitive data at risk. To make matters worse, CSRF is a serious concern because many of them do not use anti-CSRF tokens correctly, thus allowing malicious users to conduct unauthorized actions against applications [11].

Another key concern is the insecure use of APIs. Many Low-Code Developers do not secure API endpoints. On most of these platforms, the APIs are mainly exposed to attacks from the outside with weak authentication and encryption, including Microsoft Power Apps and ServiceNow [14]. The weakness in the system has cropped up because these different systems compromise on full-scale integration of robust security protocols just to attain fast development [13]. Insecurely designed APIs without strong encryption and proper authentication make those applications very prone to breaches.

By contrast, the Mendix and OutSystems platforms do have stronger measures for such risks. For example, Mendix places more emphasis on secure coding and offers built-in security features where code injection or XSS attacks can be barely realized [9], [10]. On the contrary, some platforms, such as Microsoft Power Apps, have been criticized for the way vulnerabilities are handled in an environment whose principal users are non-professional developers. Application security risks consistently emerge in the literature, particularly concerning platforms that prioritize ease of use over comprehensive security protocols [7].

Application security risks often stem from poor coding practices or inadequate security measures. Platforms like Mendix and OutSystems mitigate these risks with automated security features, unlike more vulnerable alternatives [7].

Knowledge of those vulnerabilities-insecure input handling, poor API security, weak authentication, and issues with manual configuration-provides the key to the mitigation of application security risks in LCDPs. It would be worth turning to secure coding practices, strict testing frameworks, and efficient app-level security protocols as one of the ways to prevent security breaches in applications built on top of LCDPs and protect sensitive information.

B. Information Security Risks

The most critical information security risks in LCDPs pertain to sensitive data protection, which is most often conceded through poor encryption practices or weak access controls. Data breaches remain a very huge risk, with major incidents including the recent exposure of records running into millions in both Microsoft Power Apps and Oracle due to misconfigurations of security controls. For example, Microsoft Power Apps, in 2021, had a severe breach where

security settings were so poorly configured that it left sensitive customer data open to unauthorized access, including records of a critical nature [7]. On the other hand, in Salesforce, there is poor encryption and weak access controls that are bound to leak sensitive information. Also, Oracle has had to deal with misconfigured security settings and the use of weak encryption algorithms-things that facilitate the exploitation of vulnerabilities and sensitive data theft [25]. This is a very serious case, considering all the essential financial and customer data in place.

Another pressing issue of encryption gaps across these platforms is Oracle and Salesforce. Despite having cutting-edge encryption techniques, both have been shown to have flaws in data encryption. They are vulnerable to breaches in confidentiality, most especially when sensitive data goes over insecure networks. Salesforce has been criticized for its encryption practices, which have either been outdated or configured improperly; therefore, data is quite vulnerable at transmission and at rest [18]. Likewise, Oracle is one of the strong encryption vendors; it has had its instances of inconsistency in these processes of encryption, especially at rest, hence leaving room for data breaches [15].

On the other hand, concerning data security, Mendix and OutSystems, introduced within the same period, have been very effective in supporting significant encryption techniques. For example, OutSystems encrypts data in transit as well as data at rest, thus developing minimal chances for data breach incidents to occur. These kinds of proactive encryption practices have indeed placed OutSystems ahead of platforms such as ServiceNow and Salesforce, which, although having implemented data encryption frameworks, have continued to witness frequent breaches of information security due to identified vulnerabilities [9].

Other emerging concerns involve unsecured API integrations, which further heighten the risks to data security. Especially, Microsoft Power Apps is especially vulnerable in this respect because of the lack of proper API authentication and encryption, which exposes critical parts of the system to unauthorized access. On the other hand, ServiceNow is afflicted by inappropriate API integrations, which may allow hackers to either modify sensitive data or intercept information [12].

Information Security Risks are classified separately because they concern the issue of sensitive data protection handled or stored by applications developed on LCDPs. The literature sets weak data encryption and poor implementation of access control as some of the major factors contributing to data breaches in such platforms as Microsoft Power Apps and Oracle, respectively [15]. The improvement in information security concerns addressing those weak points created by poor encryption algorithms, inappropriate access control, and insecure API integrations. Thus, information security allows one to consolidate resources or to amalgamate some resource pools under a common goal while focusing on specific security measures related to the encryption, classification of data, and access management will help protect sensitive information throughout the life of an application.

C. *Platform-Specific Security Risks*

Other specific platform-specific risks in LCDPs relate to the configuration and settings of each platform. This is particularly true for systems such as ServiceNow and Salesforce, where misconfigurations are common. Misconfigurations occur when platform administrators fail to set proper security parameters, leaving applications vulnerable to attacks. An example is that there have been various issues of misconfiguration vulnerabilities in ServiceNow, where wrong security configurations as poorly set permissions- result in unauthorized access to system components [14]. On its part, Salesforce had permitted laxity in permissions configuration that gave unauthorized access to sensitive data and furthered its leakage.

Second to the misconfigurations, the other risk involves delays in applying security patches. Pegasystems has been criticized for poor patching, leaving systems vulnerable to known vulnerabilities for long periods. Most of the vulnerabilities in such delayed patching are highly likely to be exploited by attackers because security gaps remain open [21]. Indeed, it was this very late application of patches that studies identified as the major factor leading to security incidents with Pegasystems, pointing out how timely patch management can ensure a low risk of exploitation [13].

By contrast, other platforms such as OutSystems and Mendix have taken more proactive steps regarding the patch management of their systems and configuration. Both platforms provide mechanisms for automated patching, along with detailed configuration guidelines that assist administrators in locking down their systems effectively. While this is already secured, with automated security updates, large businesses cut most risks related to human errors that cause misconfigurations. However, patch management remains an affair of manual intervention on such platforms as Salesforce and ServiceNow; hence, these increase the chances of security gaps through human error in patching processes [8] [11].

ServiceNow and Pegasystems remain highly susceptible to security breaches due to misconfigurations and late patch management. Much of this predisposition comes from dependency-oriented configuration settings set up by hand, increasing the chances of errors. Poor practices in patching, especially at Pegasystems, further add to these vulnerabilities by leaving the platform open to the attacks of hackers for a pretty long time. To minimize such risks, organizations should use automated patch management, observe strict rules of configuration, and audit on a routine basis. It should have automation for processes wherever possible. This way, human errors will be reduced, and also system security will become stronger to prevent different known exploits and vulnerabilities.

D. *User-Related Security Risks*

Users-related risks cover people's actions or behavior while using LCDPs. Among them, the most important risks are weak user credentials and insider threats. Poor credentials include weak passwords or missing multi-factor authentication, which leave applications open to unauthorized access. Salesforce and Oracle have been haunted for years by improper password policies and poor use of MFA, making these companies highly exposed to account compromise. For instance, weak password policies without MFA are common in Salesforce because users seldom follow through on simple security practices such as strong passwords and two-factor authentication [19]. This is just as similar to the systems of Oracle for credential management, which are pretty shallow. Attackers exploit weak passwords and poor login credentials so well that they further compromise security.

Along with weak credentials, insider threats are another concern that keeps on increasing when the cloud platforms give access to sensitive systems to non-professional developers. Insider threats are defined as the act of deliberately or inadvertently compromising an application by an authorized user [7]. For instance, Salesforce is used by thousands of large organizations around the world with a very heterogeneous user base, which makes it more vulnerable to insider threats. The gravest vulnerability of Salesforce would relate to poor monitoring and access control by privileged users, which may increase the likelihood of an insider threat going unnoticed. This is compounded by the fact that most security efforts focus on external threats and not on the internal vulnerabilities that may arise from misuse or accidental breaches by authorized users [19].

On the other hand, Mendix and OutSystems have been more proactive concerning user-related risks. Both ensure that strict password policies are in place and provide options for multi-factor authentication, thus keeping user accounts secure. Along with security, Mendix and OutSystems have detailed monitoring and logging of user activities, allowing administrators more power to identify and perform counteractive actions against insider threats. This level of monitoring of the users helps in decreasing the possibility of internal breaches of security and any insider threats are noticed well in time and managed with rapidity [19].

Salesforce and Oracle have lagged most in security vulnerabilities related to credential management and user monitoring. Poor credential management, weak password practices, and an inability or low rate of multi-factor authentication are very well reported in both, increasing the risk of insider threats and unauthorized access in environments where non-professional developers use these platforms frequently. It means that security is human-centered, and thus entails user training, proper mechanisms of authentication, and good monitoring to minimize risks that users can cause.[15].

IV. Discussion

This comparative analysis of Low Code Development Platforms gives a mixed picture of Mendix and OutSystems have implemented strong security frameworks, while others lag. These frameworks involve strong encryption standards, automated patch management, and proactively mitigating user-related risk vectors such as weak credentials and insider threats. This is because these platforms are usually more secure and less exposed to the vulnerabilities that are presented in the literature. Their proactive attitude towards secure coding, data protection, and monitoring of user activity reduces any security breach to a minimum.

Contrarily, the rest of the platforms, such as Microsoft Power Apps, Salesforce, and Oracle, present more serious risks regarding security breaches. These factors affect application security and information security. For instance, Microsoft Power Apps have been linked to different security breaches, most of which were linked to misconfigurations and poor encryption practices. This is further worsened by the fact that dependence on unprofessional developers increases the risk, mainly because these developers might be very unfamiliar with secure configurations. Though widely used in the enterprise space, Salesforce still suffers from continuous user-driven vulnerability exposures and insider threats. It has been more vulnerable to insider security breaches, especially due to poor credential management and insufficient monitoring of privileged users.

Of course, Oracle has equally faced huge challenges, especially those touching on data encryption and access control. While it is an acknowledged leader in advanced security features, poor encryption practices, and misconfigured security settings have exposed sensitive data to possible breaches. While Oracle and Salesforce are very relevant in enterprise contexts, their security frameworks are not as mature as those of Mendix and OutSystems.

On the other hand, both ServiceNow and Pegasystems face most of their challenges related to platform-specific risks. Thus, both depend a lot on manual security processes that increase human error and misconfiguration. Precisely, ServiceNow is very vulnerable to misconfigurations regarding API security and permissions that cause unauthorized access and data leakage. It also criticized Pegasystems for laggy patching, exposing the platform to a known vulnerability for quite a long period. This is the type of slow patch management that increases the window of risk for exploitation and makes Pegasystems an attractive target for attackers.

While both Mendix and OutSystems have automated a great deal of their security processes, such as patch management and system configurations, it drastically reduces the chances of potential platform-specific vulnerabilities. Automation minimizes the possibility of human error and guarantees that patches will be applied in a much shorter cycle, thus closing the security gaps before they can be attacked.

In this analysis, there are four clear categories of security risks: Application Security Risks, Information Security Risks, Platform-Specific Risks, and User-Related Risks. Application Security Risks normally emanate from insecure coding practices and vulnerable APIs. Major causes of information security risks are poor encryption and poor access control. For the most part, misconfiguration and late patching characterize the Platform-Specific Risks, while weak credentials and insider threats are characteristic User-Related Risks.

This structured approach lets an organization understand what kind of interventions are suitable for what type of risks, further fortifying the security of applications developed on Low-Code Development Platforms. This, in turn, means that organizations making use of such platforms, including the likes of Microsoft Power Apps, Salesforce, and Oracle, need to spend extra effort on enhancing their encryption practices, patch management, and mechanisms of user authentication. Simultaneously, the need of the hour for ServiceNow and Pegasystems is to eradicate manual processes by embracing automated security solutions with strict configuration controls.

While Mendix and OutSystems set a great example in terms of how security risks can be mitigated, mostly through proactive and automated measures, other platforms have to catch up with the hardening of the security framework. Now is the time for properly understanding and classifying these risks to enable organizations to establish appropriate best practices and ensure security in the low-code development environment.

The most common challenges for organizations regarding implementing security strategies in LCDP include a lack of awareness, constrained resources, or resistance to automation-

a special concern in non-professional developer environments. For example, automated patch management might be challenging for small organizations to use, thus leaving platforms like Pegasystems open to known risks. This integration of AI/ML has the potential to bring in predictive analytics, adaptive threat detection, and automated remediation. These technologies are for improving encryption, automating configuration management, and insider threat monitoring to handle vulnerabilities in Oracle and Salesforce platforms, among others.

Future Research on the applications of AI/ML can develop more resilient and scalable low-code platforms to proactively handle risks in an ever-evolving security landscape.

V. Conclusion

This research has performed an in-depth analysis of the security risks of low-code development platforms, coupled with their underlying vulnerabilities because of their increasing adoption, especially by non-professional developers. Mendix and OutSystems have focused on robust security measures that include strong encryption, automated patch management, and user risk mitigation, while Microsoft Power Apps, Salesforce, and Oracle still pose greater challenges in these respects. These platforms are more vulnerable to application security risks, such as code injection and cross-site scripting, and to information security risks like data breaches and poor encryption practices. ServiceNow and Pegasystems have the highest platform-specific risk driven by reliance on manual security processes and delayed patching that leads to misconfigurations, with potential exploitation. This analysis underlines the need for a multidimensional approach with technology solutions such as automation, and secure coding, but also human-oriented measures of training and strong credential management.

Organizations should, therefore, identify the different layers of risk application, information, platform-specific, and user-related-to tailor their security interventions accordingly. Improved encryption, automation of security-related activities, and authentication at the user end are some ways organizations can secure their LCDP-based applications. In days to come, platforms that are currently behind in terms of security, such as Microsoft Power Apps and Salesforce, need to strengthen their security frameworks to prevent future breaches. Only through continuous improvement and the use of best practices can one continue to develop an even more secure and future-proof LCDP environment.

References

[1] M. Woo, "The Rise of No/Low Code Software Development—No Experience Needed?," 2020.

[2] Grand View Research, "Low-code Development Platform Market Size, Share & Trends Analysis Report By Application Type (Web-based, Mobile-based), By Deployment Type (Cloud, On-premise), By Organization Size, By End-use, By Region, And Segment Forecasts, 2023 - 2030," Grand View Research, Inc., San Francisco, California, USA., 2023.

[3] Md Abdullah Al Alamin Gias Uddin· Sanjay Malakar · Sadia Afroz ·, "Developer discussion topics on the adoption and barriers of low code software development platforms," 8 November 2022.

[4] Tetsuro Miyake, Yoshimasa Masuda, Akiko Oguchi & Atsushi Ishida, "Strategic Risk Management for Low-Code Development Platforms with Enterprise Architecture Approach: Case of Global Pharmaceutical Enterprise," 31 May 2023.

[5] Gómez, J., et al., "Advanced Security Capabilities in Oracle's Low-Code Development Platforms.," *Journal of Cloud Computing and Security,* pp. 15(2), 123-145, 2023.

[6] Lee, S., & Jones, M., "Innovative Use of Case Management and Decisioning in Pegasystems.," *International Journal of Business Process Management,* pp. 10(4), 200-220, 2023.

[7] V., Sureshkumar., B., Baranidharan, "A study of the cloud security attacks and threats.," 2021.

[8] N. Paavo, "Emerging low-code/no-code paradigm: Evaluating adoption, opportunities, and cyber security challenges in the information technology sector," 2023.

[9] Lourenço Miguel, Gasiba Tiago Espinha, Pinto-Albuquerque Maria, "You Are Doing it Wrong - On Vulnerabilities in Low Code Development Platforms," 2023.

[10] P. Laura, "Implementing a Mendix low-code application using best practices," 2022.

[11] O. Alkisti, "Safety Risks and Security Threats in Low-code Software Development," p. 17, 2022.

[12] Wal, Niels van der, "Exploring Change Impact Analysis in Low-Code Development Platforms," 2022.

[13] N. M. A. Pulido, "Applying Behavior Driven Development Practices," 2019.

[14] T. Nechyporenko, "ServiceNow as a platform – practical research," 2015.

[15] Baggia Alenka, Leskovar Robert, Rodič Blaž, "LOW CODE PROGRAMMING WITH ORACLE APEX OFFERS NEW OPPORTUNITIES IN HIGHER EDUCATION," 2019.

[16] S. Spets, "Application Security Verification Standard Compliance Analysis of a Low Code Development Platform," 2022.

[17] K. S. Sharanya, "HANDLING WORK FROM HOME SECURITY ISSUES IN HANDLING WORK FROM HOME SECURITY ISSUES IN SALESFORCE SALESFORCE," 2023.

[18] Jahan Sadaf, Ahmad Faiyaz, "Ensuring Data Security on Salesforce: A Comprehensive Review of Security Measures and Best Practices," 2023.

[19] Jahan Sadaf, Ahmad Faiyaz, Haroon Mohd, "Protecting Sensitive Data in Salesforce-A Multi-Layered Security Approach," p. 13, 2023.

[20] M. Mike, "Overcoming the risks of privileged user abuse in Salesforce," *Publisher: Elsevier Ltd,* p. 8, 2018.

[21] August Terrence, Niculescu Marius Florin, Shin Hyoduk, "Cloud Implications on Software Network Structure and Security Risks," 2018.

[22] L. Jens, "Best Practices for Microsoft Power Apps Implementation," 2024.

[23] Frank Ulrich, Maier Pierre, Bock Alexander, "Low code platforms: Promises, concepts and prospects. A comparative study of ten systems," *Universität Duisburg-Essen, Institut für Informatik und Wirtschaftsinformatik (ICB), Essen,* 2021.

[24] H. d. With, "Driving citizen development through the enterprise-wide deployment of low-code development platforms," 2022.

[25] C. T. Rao, "Enhancing Cloud Security with Oracle Cloud Security Applications," p. 21, 2024.

[26] "Low/No Code Development in 2024: How It Works & Use Cases," Jan 11. [Online]. Available: https://research.aimultiple.com/low-code/.

[27] Yajing Luo1, Peng Liang1∗, Chong Wang1, Mojtaba Shahin2, Jing Zhan, "Characteristics and challenges of low-code development: The practitioners perspective".